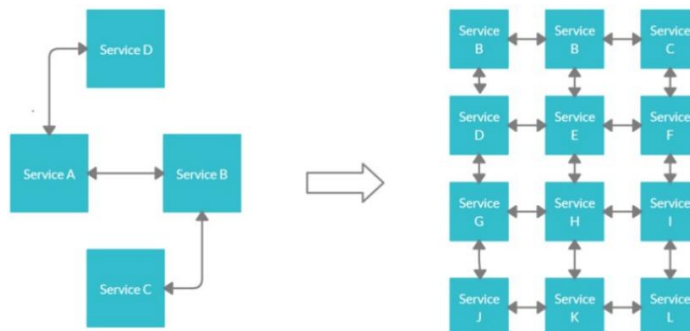
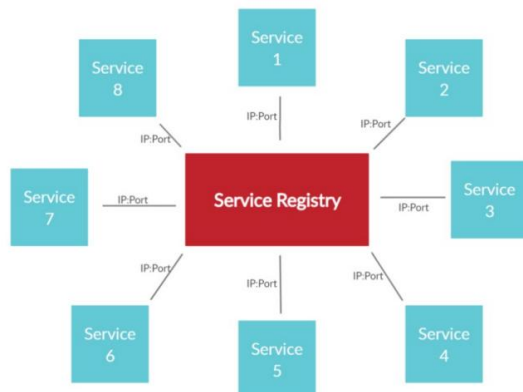


- Microservices are an architectural style where applications are developed as a collection of small, loosely coupled, independently deployable services.
- Comparison with Monolith
 - ⌘ Monolith: tightly coupled, single code-base, single point of failure
 - ⌘ Microservices: loosely coupled, multiple independent services
- Rest Client, Open Feign => these are some tools for api calling from one service to another.
- **Service Discovery**
 - ⌘ In microservices, each service has to talk with the others to function as a *complete application*.
 - ⌘ For this, they need to know each other's *network locations*.
 - ⌘ Service discovery comes into play here, maintaining a record of service's locations, helping each other find each other and enabling communication.
- **Service Registry**
 - ⌘ If we add one more service in our Microservice Architecture, all the services have to keep the address of new service.
 - ⌘ If we add multiple services, then it'll not be properly maintainable.
 - ⌘ So, instead of keeping the address themselves, all the services register with a *Service Registry* so that if any new service is introduced, it can know about the other's addresses.
 - ⌘ It's a *Centralized* service that registers all the services.



(without service registry)



(with service registry)

➤ Spring Cloud Eureka

- ⌘ It's a REST based service which is primarily used for acquiring information about services that you would want to communicate with.
- ⌘ This REST service is also known as *Eureka Server*.
- ⌘ The services that register in Eureka Server to obtain information about each other are called *Eureka Clients*.

➤ Eureka Server in Spring Boot

- ⌘ Its another spring application service having *Eureka* dependency which will register all the services.

```

<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
  
```

- ⌘ Enable the Eureka server

```

@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServiceApplication {
  
```

- ⌘ Disable the service registering for eureka service (by default it is enabled)

```

eureka:
  client:
    fetch-registry: false
    register-with-eureka: false
  
```

- ⌘ In the micro-services, (because these are the eureka clients) add the *Eureka Client* dependency.

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
</dependencies>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

- ⌘ Register the eureka server in the client services

```
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
```

- ⌘ Now when you run the eureka server then run the other services (eureka clients) they'll get registered in the eureka server. You can see in the eureka dashboard (localhost:8761 or whatever port you have configured)

➤ **How Eureka Works?**

⌘ **Service Registration**

- ⌘ A service registers itself with the Eureka Server upon startup.

⌘ **Heartbeat**

- ⌘ The service sends heartbeats periodically to renew its lease with the Eureka Server.
- ⌘ Otherwise Eureka will think that the service is down.

⌘ **Service Discovery**

- ⌘ Other services can query Eureka server to discover the location (IP and Port) of the registered service.

⌘ **Health Check**

- ⌘ Eureka performs health checks to ensure that registered services are still healthy.

⌘ **Eviction**

- ⌘ If a service stops sending heartbeats and its lease expires, the Eureka evicts it from the registry.

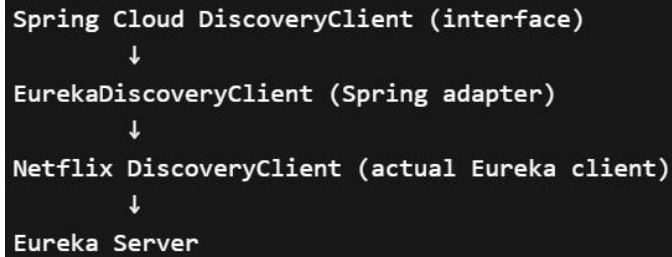
⌘ The auto-configure class for Eureka Server is *EurekaServerAutoConfiguration* .

- ⌘ This manages service registration, service registry, heartbeats, replication between servers.

⌘ There is a *singleton* class *PeerAwareInstanceRegistryImpl* which is responsible for sending *heartbeat* to the Eureka Server.

➤ How to use Eureka?

- There are 2 same interfaces present, named as **DiscoveryClient**, one belong to *spring cloud* and another belongs to *netflix*.
- The flow of call happens like below



- EurekaDiscoveryClient* (by *netflix*) implements *DiscoveryClient* (by *spring cloud*)
- EurekaDiscoveryClient* contains an object of type *EurekaClient*
- This *EurekaClient* is implemented by *DiscoveryClient* (by *netflix*).

```
List<ServiceInstance> instances = discoveryClient.getInstances( serviceId: "order-service");
```

this *serviceId* is the name of spring boot application that is being mentioned in *application.yml* or *application.properties* file.

- NOTE:** You may see *RestClient* in Eureka classes but it doesn't use *RestClient*.

It uses its own HTTP client internally. The netflix client uses *EurekaHttpClient*

```
@GetMapping("/fetchOrders")
public String fetchFromOrderService() {
    ServiceInstance orderService = discoveryClient.getInstances( serviceId: "order-service").getFirst();

    return restClient.get() RequestHeadersUriSpec<capture of ?>
        .uri( uri: orderService.getUri() + "/api/v1/orders/helloOrders") capture of ?
        .retrieve() ResponseSpec
        .body( bodyType: String.class);
}
```

This is how you can implement microservices.

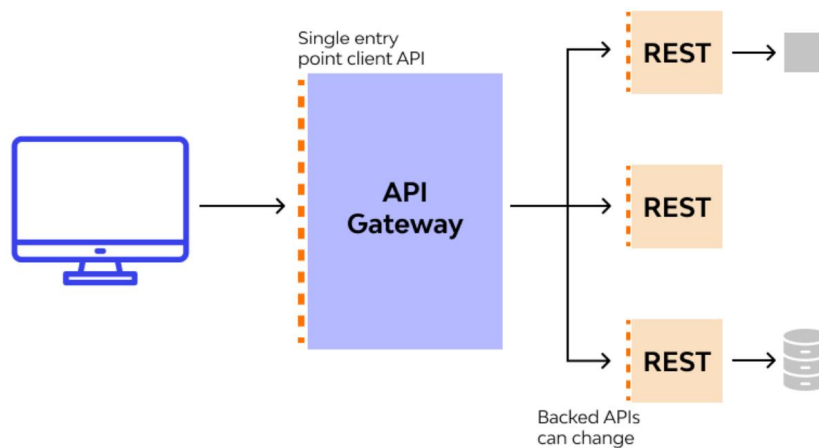
Eureka is only for the IP and Service management. RestClient is to use that url and trigger HTTP requests and get response.

```
spring:
  application:
    name: <your-service-name> # Name of your microservice
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/ # URL of the Eureka Server where the client should register
    fetch-registry: true # (usually true for clients)
    register-with-eureka: true # (usually true for clients)
  instance:
    prefer-ip-address: true # Use IP instead of hostname for registration (useful for containers)
    lease-renewal-interval-in-seconds: 10 # (default is 30)
    lease-expiration-duration-in-seconds: 30 # (default is 90)
```

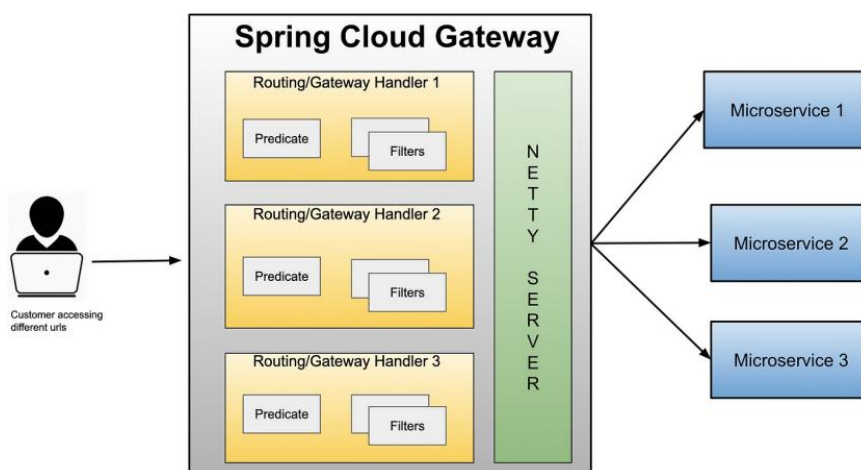
Spring Cloud API Gateway

➤ Description

- ⌘ APIs are a common way of communication between applications. In case of microservice architecture, there will be a number of services and the client has to know the host names of all underlying applications to invoke them.
- ⌘ To simplify the communication, we prefer a component between client and server to manage all API requests called API gateway. Additionally, we can have other features which includes:
 - ⌘ *Security*: authentication, authorization
 - ⌘ *Routing*: routing, request/response manipulation, circuit breaker
 - ⌘ *Observability*: metric aggregation, logging, tracing



- ⌘
- ⌘ There are multiple options out there, but the most optimized in case of Spring is Spring Cloud Gateway.



⌘

- Implementation of API gateway

- ⌘ Keep 2 dependencies: Eureka Discovery Client, Gateway (Spring Cloud)
(Eureka Client because API gateway also needs to connect with the Eureka server to get the details about other services)
- ⌘ If you trace the *pom.xml* dependencies
 "spring-cloud-starter-gateway-server-webmvc" ---- "spring-cloud-gateway-server-webmvc" ---- "spring-cloud-loadbalancer" ---- .

➤ **Spring Cloud Gateway route**

- ♣ The destination that we want a particular request to route to.
- ♣ It comprises of destination URI, a condition that has to satisfy - Or in terms of technical terms, *Predicates*, and one or more *filters*.

➤ **Spring Cloud Gateway predicate**

- ```

^ This is literally a condition to match, I.e. kind of “if” condition.
^ - Path=/api/v1/orders/**
^ - Method=GET
^ - Header=User-Agent, Mozilla/*

```

► **F**

**► F**

► **F**

➤ **F**

➤ **F**

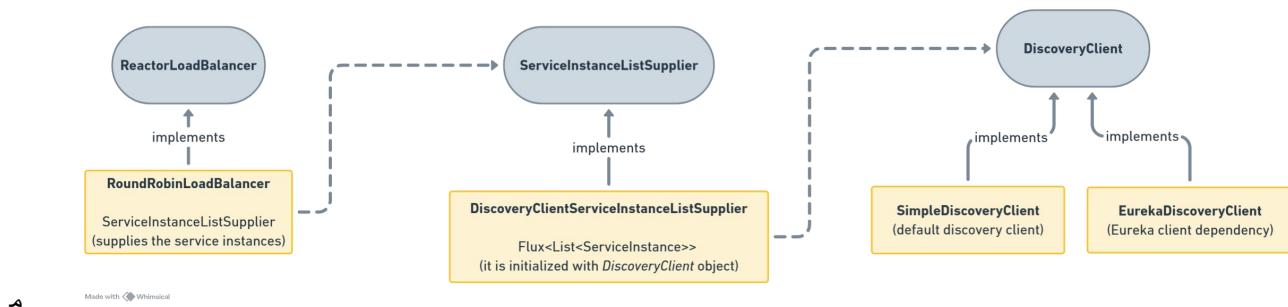
➤ **F**

► **F**

➤ **F**

► **F**

- Some confusion regarding Spring Cloud API Gateway
  - ⌘ In case of oauth or payment integration or caching (redis or something else), etc.. spring has library which delegates the calls to the specific services.
  - ⌘ But, Spring Cloud API Gateway is a stand alone service that implements API gateway and integrated Load Balancer.
    - ⌘ Just like **nginx** provides API gateway and load balancer, Spring Cloud also provides the same **by its own**.
  - ⌘ Without *Eureka Client* dependency also it'll work. But it'll not be dynamic.



## ➤ Working of Gateway and Load Balancer

- ⌘ Lets say you have configured the application.yml file like below

```

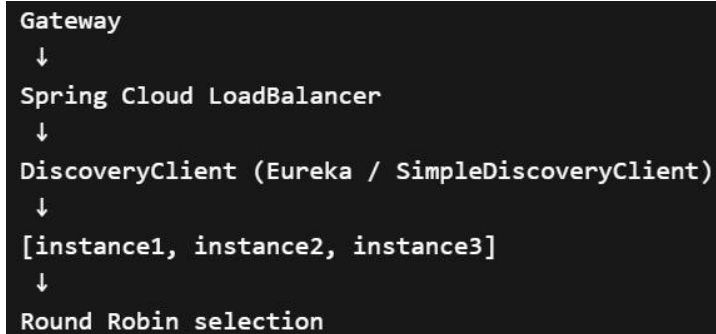
spring:
 cloud:
 discovery:
 client:
 simple:
 instances:
 ORDER-SERVICE:
 - uri: http://localhost:8081
 - uri: http://localhost:8082

```

- ⌘ In this case you have directly mentioned the end points of the instances of order-service (same service can have multiple instances)
- ⌘ [spring docs for this yaml structure \(its present in .properties format\)](#)  
`spring.cloud.discovery.client.simple.instances.service1[0].uri=http://s11:8080`
- ⌘ So, here the **SimpleDiscoveryClient** will return all these instances to the **RoundRobinLoadBalancer**, out of which it'll choose a instance and delegates the request to that.  
 (**RoundRobinLoadBalancer** class has a **choose()** method)
- ⌘ Internally it calls like **lb://ORDER-SERVICE** which triggers the load balancer.



- Here the flow will be like *client -- gateway --- discovery client --- instances --- choose (load balance) --- delegate to instance*



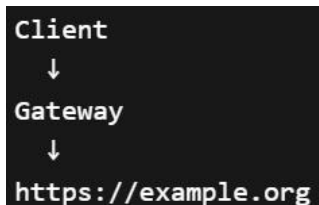
- In case of the below, load balancing will not happen and you can only give one end-point per service

```
spring:
 cloud:
 gateway:
 routes:
 - id: after_route
 uri: https://example.org
```

```
routes:
 - id: users
 uri: http://user-service:8081
 predicates:
 - Path=/users/**

 - id: orders
 uri: http://order-service:8082
 predicates:
 - Path=/orders/**
```

- Its flow will be like *client -- gateway -- delegate to instance*



- Now, if you are using **Eureka Client** then you don't need to hard-code the end points, because instead of **SimpleDiscoveryClient** here **EurekaDiscoveryClient** will be used which will get the end points of the instances from the Eureka Server.

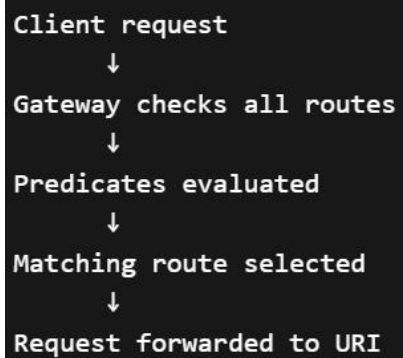
```

spring:
 cloud:
 gateway:
 routes:
 - id: order-service
 uri: lb://ORDER-SERVICE
 predicates:
 - Path=/api/v1/orders/**
 filters:
 - AddRequestHeader=X-Custom-Header, Anuj
 - id: inventory-service
 uri: lb://INVENTORY-SERVICE
 predicates:
 - Path=/api/v1/inventory/**
 - id: no-service
 uri: lb://NO-SERVICE
 predicates:
 - Path=/api/v1/no-service/**
 filters:
 - AddRequestHeader=X-Custom-Header, Anuj
 - RedirectTo=302, https://codingshuttle.com

```

### ➤ predicates

- ⌘ Its nothing but a condition, if it passes then only the request will be forwarded otherwise not.
- ⌘ In case of *predicates* **Path**, if the request end point matches this path, then only the request will be forwarded otherwise not.
- ⌘ For example:
  - \* **Path=/orders/\*\***
  - \* if request path is /orders/123 then it'll match and it'll be forwarded.
- ⌘ Different types of predicates are there like Path, Method, Header, Cookie, Host ..etc



- ⌘ Don't think it is used for authorization, for example

*Cookie=mycookie,mycookievalue* , it'll just check if cookie having name

“mycookie” is present and having the value “mycookievalue”, if this condition passes then it’ll forward. It’ll not validate the cookie value.

- To configure the API Gateway

```
cloud:
 gateway:
 routes:
 - id: order-service
 uri: lb://ORDER-SERVICE
 predicates:
 - Path=/orders/**
 - id: inventory-service
 uri: lb://INVENTORY-SERVICE
 predicates:
 - Path=/inventory/**
```

⌘

- ✶ order-service has context path: /orders, and the main order controller has path “core”;  
so, the path is “/orders/core”
- ✶ Similarly inventory-service has path: “/inventory/products”
- ✶ So, this will work. If we open *localhost:8080/orders/core* or something like that, then it’ll forward the request to the specific service.

- ⌘ But, sometimes we want versioning, like “/api/v1” or “/api/v2”; but we don’t want to give these as context path in the respective services. We want to control these from gateway itself.

```
cloud:
 gateway:
 routes:
 - id: order-service
 uri: lb://ORDER-SERVICE
 predicates:
 - Path=/api/v1/orders/**
 filters:
 - StripPrefix=2
 - id: inventory-service
 uri: lb://INVENTORY-SERVICE
 predicates:
 - Path=/api/v1/inventory/**
 filters:
 - StripPrefix=2
```

✶

- Here, we can use the filter *StripPrefix*, 2 means it'll remove the first 2 i.e. "api" and "v1" and forward the remaining i.e. [http://localhost:<service\\_port>/orders/\\*\\*](http://localhost:<service_port>/orders/**) (it'll not include /api/v1 )
- Filters are executed between gateway and services.
- Filters are used to modify the request at gateway level before forwarding this to the services.

- F
- F
- F
- F
- F
-